



Stage 1 Build a working coding harness

Building our own Claude Code/Codex harness

Build a small application that takes a coding task, uses tools to change a repository, runs checks, and shows the result. This is the first of three working deliveries of your semester project.

A coding harness is the application around an LLM. It connects the model to tools and controls how those tools run. A proof of concept, or POC, is a small version that works from start to finish.

Your first working version

A user submits a bug-fix request through a CLI or UI. Your controller asks the model what to do, checks each request, runs allowed tools, and returns their results. The user receives the changed files, test results, and a diff.

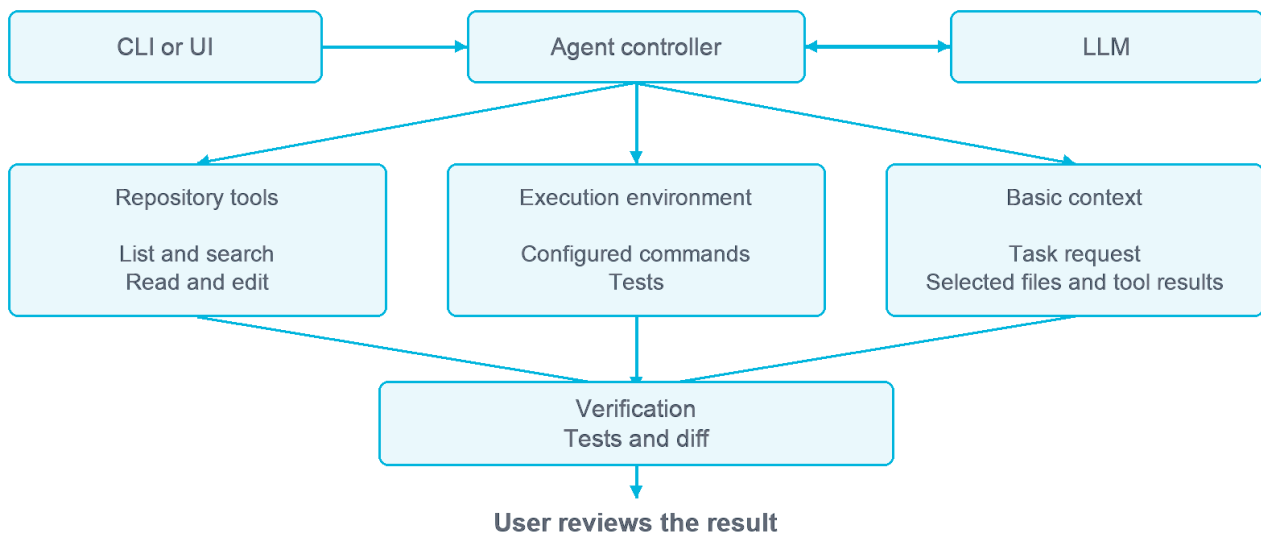


Figure 1 Stage 1 connects the model to tools and basic context, then checks the result.

Choose your implementation

Build your own coding harness in Python. You may use an agent framework and libraries, and reuse your lab code. You design, build, and test the application. Do not start from an existing coding harness. A clear CLI can earn full credit; a UI is also welcome.

Local Ollama is a suitable starting point. A paid service is not required.



What to build

Part	Required behavior
Setup	Provide dependencies, example settings without secrets, and a working launch command.
Interface	Accept a task and show progress, changed files, the diff, and check results.
Controller	Send context to the model, validate tool requests, execute allowed actions, and return results or errors.
Repository tools	List, read, search, and edit files within the allowed repository copy.
Execution	Run configured commands and tests in a contained environment. Capture output and exit codes.
Limits	Enforce action and output limits. Count denied actions and retries. Return clear errors and stop safely.
Verification	Check the resulting code. Keep failed or unavailable checks visible even if the model says it is done.

Keep responsibilities clear

The example below shows one possible structure. A framework may provide several of these parts. Use classes, functions, or both; you do not need to copy these names.

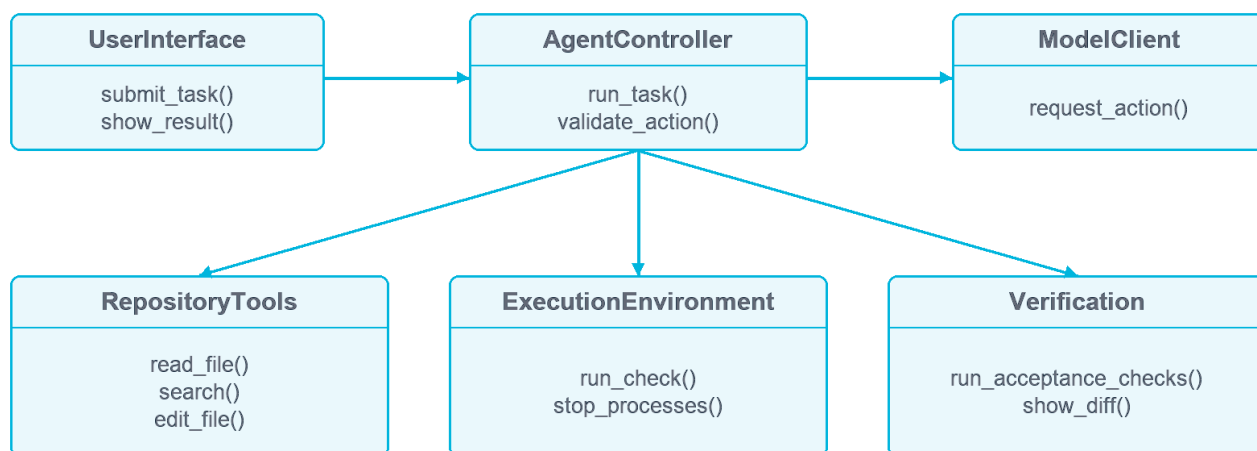


Figure 2 Example class responsibilities. Arrows mean that one component uses another.

The execution environment contains commands and tests. File tools must enforce the same allowed scope. A class name alone does not create a security boundary.



Make the loop work

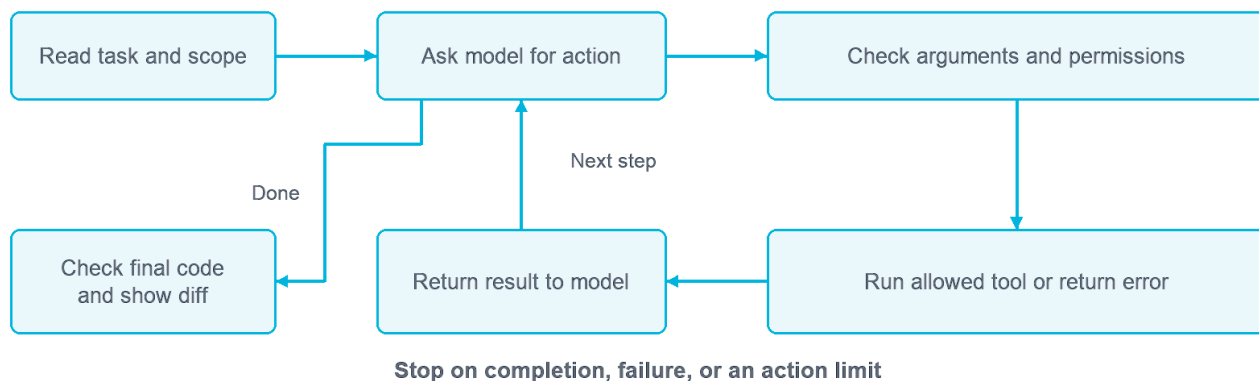


Figure 3 Tool results return to the model. Completion leads to final checks and the diff.

Protect the working environment

Use a disposable copy of the target repository. Keep credentials and unrelated host files outside executed code. Run repository code and tests inside a contained environment; an existing sandbox setup is allowed. A Git worktree alone is not a sandbox.

Check tool arguments and permissions in application code. Treat repository text and model replies as untrusted input. Keep push, merge, and deployment disabled. A stuck command must be stoppable without leaving child processes or further writes behind.

Tests to include

Test	Expected result
File tools	Read, search, and edit the intended files. Reject a path outside the allowed area.
Controller	A scripted model reply calls the correct tool and receives its result.
Invalid request	Unknown tools or invalid arguments produce a clear error without executing an action.
Failed command	A nonzero exit code and its output stay visible. The run does not claim that checks passed.
Action limit	A repeating scripted reply reaches the limit and stops further actions.
Output limit	Large output is bounded and marked as shortened.
Bug fix	The acceptance check fails on the starting code and passes after the change. Existing tests still pass.

Use scripted replies for repeatable controller tests. Core tests must run without a live account or API key. Use a real model for the coding demonstration.



Choose a feature

Choose one of these repositories, or choose another public Python repository yourself:

- [OpenStock](#) — Flask and SQLite; stock, invoicing, and permissions.
- [Inventory Management System API](#) — FastAPI and MongoDB; a larger API project.
- [FastAPI RealWorld Example App](#) — FastAPI and PostgreSQL; users, articles, comments, and tests.
- [Cosmic Python example application](#) — domain logic, services, message handling, and many tests.

The repository must run locally, have tests, include at least two modules that work together, and contain real business rules.

Keep the task small enough to complete as a working POC, but make it involve behavior across modules.

Prepare the acceptance check

1. Record the target repository and exact starting commit. Write the bug-fix/feature request and state which files or behavior may change.
2. Create a check to reproduce the behavior. Confirm it fails on the starting code.
3. Keep this acceptance check outside the agent's writable area. Run it against the resulting code in contained execution.
4. Run the harness with a real model. Show the diff, the passing acceptance check, and existing regression tests. Identify any manual help.

Delivery 1 checklist

- Runnable source code via repository link and commit mention if needed.
- Secrets if needed to run the project.
- Automated tests for the tools, controller, errors, and limits.
- A short README and diagrams that match your implementation and how to run it.
- A demo video no longer than 4 minutes.

Submit a working application.